MODULE 28 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 2 of 3

Twenty new questions. Three of them deliberately invert a question from paper one.

QUESTIONS 20 HOLD 9 seconds PAPER 2 of 3

admin@sthithapragna.com

> A team is arguing for adopting Infrastructure as Code over provisioning through the cloud console.

What is the headline advantage?

A. Resources become cheaper to run

B. Configuration can be versioned, reused and shared

C. Infrastructure becomes automatically highly available

D. Credentials are secured by the tooling

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the headline advantage?

A Cheaper

The same instance costs the same however it was provisioned. Cost is a property of what you built.

B Version, reuse, share

Correct, and it is HashiCorp's own phrasing for the advantage of code over a graphical interface.

C Highly available

Availability comes from architecture. Declaring it in code does not create it.

D Credentials secured

Backwards. Code creates a new way to leak credentials, which is why secrets never belong in a repository.

ON THE EXAM

Domain 1. Every wrong option here is a genuine benefit of something else, which is what makes them attractive.

> A director tells the board that adopting Terraform will make it impossible for engineers to change infrastructure outside the approved process.

How should this be corrected?

A It is accurate once state is stored remotely

B It is accurate for immutable resources only

C It is accurate once the configuration is in version control

D Terraform surfaces such changes as drift, but cannot prevent them

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should this be corrected?

A **Remote state fixes it**

Remote state solves sharing. It has no authority over what somebody does in a cloud console.

B **Immutable only**

Invents a rule. Immutability describes how you choose to change things, not whether the provider accepts edits.

C **Version control fixes it**

Adds history and review to the configuration, and reaches nothing outside it.

D **Detects, cannot prevent**

Correct. Prevention needs provider-side permissions or policy. Terraform reports the difference; it has no authority to stop it.

ON THE EXAM

Domain 1. IaC detects unmanaged change. Preventing it means taking away the permission to make it.

> A configuration must install a specific provider version, and separately must configure that provider with a particular region.

Where does each belong?

A Both in the provider block

B Both in required_providers

C Version in required_providers; region in the provider block

D Version in the provider block; region in required_providers

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Where does each belong?

A Both in provider

The provider block configures behaviour. It does not decide which version is installed.

B Both in required_providers

That block names the plugin and constrains its version. Region is a provider-specific setting, not part of a plugin's identity.

C Version then region

Correct. required_providers declares and installs; the provider block parameterises what was installed.

D Reversed

Inverts both. This is the exact confusion the question is testing.

ON THE EXAM

Domain 2. Declaration installs, configuration parameterises. Two blocks, two jobs, constantly conflated.

admin@sthithapragna.com

A team needs a provider published by a third-party vendor, listed in the Terraform Registry as a Community provider.

How is it installed?

A By terraform init, exactly like an official provider

B By downloading the binary and placing it manually

C Only through a private registry

D By terraform init, but only with an explicit -upgrade

◆ PAUSE AND DECIDE

How is it installed?

A init, like any other

Correct. Since 0.13, official, partner and community providers all install automatically from the registry.

B Manual placement

True before 0.13 and false since. This is the most persistent out-of-date claim about providers.

C Private registry only

Community providers are on the public registry. A private registry is for things you host yourself.

D Needs -upgrade

-upgrade re-resolves versions against constraints. It is not required for a first installation.

ON THE EXAM

Domain 2. The tier describes who supports a provider, not how it installs. All three tiers install identically.

> A pipeline runs on pull requests in an environment with no cloud credentials, by design. It must reject a change that references an argument name which does not exist on a resource type.

What should it run?

A terraform plan, granting it read-only credentials

B terraform fmt -check -recursive

C A check block asserting the argument exists

D terraform init -backend=false, then terraform validate

◆ PAUSE AND DECIDE

What should it run?

A plan with credentials

Would catch it, and grants credentials to a pipeline deliberately built without any, for a problem that never needed them.

B fmt -check

A formatter, not a linter. It will happily format a configuration referencing arguments that do not exist.

C A check block

Warns rather than blocking, so it could not reject anything, and it runs during a plan.

D init -backend=false, validate

Correct. validate checks attribute names and types using locally installed schemas, with no remote access at all.

ON THE EXAM

Domain 3. An invalid argument name is a language error. validate needs init but not a backend, and no credentials.

> A pipeline must generate a plan, have it approved by a human in a separate step minutes later, then apply exactly what was approved, with no possibility of a different set of actions running.

What supports this?

A Run plan and apply back to back with -auto-approve

B terraform plan -out, then apply that plan file

C terraform apply -refresh=false, to freeze the inputs

D terraform plan -detailed-exitcode, then a second plan

◆ PAUSE AND DECIDE

What supports this?

A Back to back

Removes the approval step entirely. The requirement is that a human approves between plan and apply.

B Saved plan file

Correct. Apply executes a saved plan exactly as written, with no prompt, which is why the two-step workflow exists.

C -refresh=false

Skips reading reality. It does not preserve a set of approved actions, and makes the plan less accurate.

D A second plan

A second plan is a new plan. What was approved and what runs could differ, which is the thing being ruled out.

ON THE EXAM

Domain 3. Paper one question 20 had a saved plan go stale over five days. Here the gap is minutes, and the saved plan is exactly right.

admin@sthithapragna.com

> A virtual machine is unresponsive after a failed in-guest upgrade. Its configuration is unchanged and correct, and Terraform plans no changes because the provider reports the instance as healthy.

What should be run?

A terraform apply -replace for that instance

B terraform apply -refresh-only

C terraform destroy, then terraform apply

D terraform state rm, then terraform apply

◆ PAUSE AND DECIDE

What should be run?

A -replace

Correct. This is the documented use: a degraded object rebuilt identically, with the replacement visible in the plan.

B -refresh-only

Records what the provider reports, and the provider reports a healthy instance. It would find nothing.

C destroy then apply

Destroys everything in the configuration to rebuild one sick machine.

D state rm then apply

Terraform forgets the instance, then builds a second one. The unresponsive machine keeps running, unmanaged.

ON THE EXAM

Domain 3. Terraform cannot see in-guest failure. -replace is how you tell it something is wrong that no API can report.

> Fourteen buckets are created from a list of team names using count. A new team is added alphabetically first. The plan proposes to destroy and recreate thirteen existing buckets.

What is the cause?

A Bucket names must be globally unique

B The list must be sorted before use

C count identifies instances by position, so every index shifted

D The provider requires buckets to be created in order

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the cause?

A Global uniqueness

True of bucket names and irrelevant. Only one name was added; the other thirteen are unchanged.

B Sort it

The list is already alphabetical, and sorting cannot rescue an identity scheme based on position.

C Positional identity

Correct. Inserting at the front shifts every index, so Terraform sees thirteen resources with different names.

D Ordering requirement

Ordering is not the problem. Terraform believes thirteen different buckets are now required.

ON THE EXAM

Domain 4. for_each over a set keys instances by value, so inserting a name affects nothing else. Appending to a list causes no churn, which is why this hides for years.

> An application instance reads its configuration from a storage object at boot only. The team wants the instance rebuilt automatically whenever that object changes. No argument on the instance references the object.

Which mechanism does this?

A depends_on referencing the storage object

B create_before_destroy on the instance

C ignore_changes on the instance's user data

D replace_triggered_by referencing the storage object

◆ PAUSE AND DECIDE

Which mechanism does this?

A depends_on

Orders operations only. The object would be updated first, and the instance would be left alone.

B create_before_destroy

Changes the order of a replacement that is already happening. It never causes one.

C ignore_changes

Suppresses reactions to change. The opposite direction entirely.

D replace_triggered_by

Correct. It replaces this resource when a referenced resource or attribute changes, which is precisely the requirement.

ON THE EXAM

Domain 4. depends_on says after. replace_triggered_by says again. Both take references, which is why they blur.

admin@sthithapragna.com

> A platform team wants to be alerted when any TLS certificate is within thirty days of expiry. The alert must appear in the run output and must never cause an unrelated deployment to fail.

Which mechanism fits?

A A postcondition on the certificate resource

B A check block with an assert on the expiry date

C A hard-mandatory Sentinel policy

D A variable validation comparing expiry to the current date

◆ PAUSE AND DECIDE

Which mechanism fits?

A — A postcondition

Blocks, and a postcondition failure also prevents changes to dependent resources. An expiry notice would become an outage.

B — A check block

Correct. A failed assertion reports a warning and the operation continues, which is exactly the stated requirement.

C — Hard-mandatory policy

Cannot be overridden by anyone, so an approaching expiry would stop every deployment until the certificate was renewed.

D — Variable validation

Runs before any resource exists, so it cannot see a certificate's expiry date, and it blocks.

ON THE EXAM

Domain 4. Paper one question 7 had a check block as the WRONG answer, because enforcement was required. Here the requirement forbids blocking.

> A module author needs to assert that a resource, once created, has a property that can only be read from the resource itself.

Which mechanism can express this?

A A variable validation block

B A precondition in the resource's lifecycle block

C A postcondition, which can use self

D A check block, which is the only mechanism with self

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which mechanism can express this?

A Variable validation

Runs when variables are set, before any resource exists. There is nothing to inspect.

B A precondition

Runs before the block is provisioned, and cannot use self. The object does not exist yet.

C A postcondition with self

Correct. It runs after provisioning and can reference the enclosing object's own attributes.

D A check block

Check blocks are top-level and do not carry self. The claim inside the option is what makes it wrong.

ON THE EXAM

Domain 4. If the condition must inspect the object itself, only a postcondition can see it. That is the practical tell.

> A pipeline exports TF_VAR_environment as staging. The repository contains a committed terraform.tfvars setting environment to production. The deployment behaves as production.

Why?

A tfvars files have higher precedence than environment variables

B TF_VAR_ names are case-sensitive and must match exactly

C Environment variables are read only when no default exists

D The variable must also be declared with nullable = false

◆ PAUSE AND DECIDE

Why?

A tfvars outranks env vars

Correct. Environment variables sit second from the bottom, above only the default. Every tfvars file beats them.

B Case sensitivity

Names are case-sensitive, and a mismatch means the variable is unset, producing a prompt or an error rather than a wrong value.

C Only without a default

Environment variables are always consulted. They simply lose to anything above them in the order.

D nullable = false

Governs whether null may be passed. It has no bearing on precedence.

ON THE EXAM

Domain 4. Most tools treat an environment variable as an override of last resort. Terraform is the opposite, which is why this is reliably missed.

> A root configuration needs the public IP of a NAT gateway that a child module creates. An engineer writes a reference to the resource inside the module and Terraform rejects it.

What is required?

A Use terraform_remote_state to read the module's resources

B Declare an output in the module, then reference it

C Move the NAT gateway into the root module

D Read the value with terraform state show

◆ PAUSE AND DECIDE

What is required?

A Remote state

Reads another configuration's root outputs. This module is in the same configuration, and outputs are still the only interface.

B Declare an output

Correct. A module's outputs are its entire public interface. Undeclared internals are unreachable by design.

C Move it to root

Dismantles the module to read one value. Modules exist so internals can stay internal.

D state show

An inspection command that prints for a human. It cannot supply a value to configuration.

ON THE EXAM

Domain 5. There is no syntax for reaching inside a module. Encapsulation here is deliberate, not incidental.

> An engineer prepares a module for the public registry. The repository is public, named correctly, follows the standard structure and has a description. She tags the release release-2026-q1. The registry accepts the repository but lists no versions.

What is wrong?

A A module needs at least three versions before any are shown

B The tag must be signed

C The repository must be named with a terraform prefix

D Release tags must be a semantic version, optionally prefixed with v

◆ PAUSE AND DECIDE

What is wrong?

A Three versions

Invents a rule. One release tag is enough, and at least one is required to publish initially.

B Signed tags

Provider releases involve signing. Module publishing has no such requirement.

C Naming

A real requirement, and the question says the repository was named correctly and accepted.

D Not a semantic version

Correct. The registry identifies versions from x.y.z tags. Anything else is ignored silently.

ON THE EXAM

Domain 5. The registry accepted the repository and found no versions. That split clears the naming rule and points at the tags.

> A team converts nine production databases from count to for_each, with unchanged names. The plan proposes to destroy all nine and create nine replacements. The refactor must touch no infrastructure.

What should they add?

A. A moved block per instance, mapping each old index to its new key

B. prevent_destroy on each database

C. ignore_changes so the address change is not detected

D. A removed block per database, then re-import each one

◆ PAUSE AND DECIDE

What should they add?

A moved blocks

Correct, and exactly what moved exists for. One per instance, because what changed is how instances are keyed.

B prevent_destroy

Blocks the plan rather than fixing it. Terraform would refuse to apply and the refactor would stay stuck.

C ignore_changes

Ignores attribute changes, not addresses. Instance identity is not an attribute.

D Remove then re-import

Reaches the same state by a longer route, with a window where nine production databases are unmanaged.

ON THE EXAM

Domain 6. Terraform compares addresses, not intentions. A refactor that changes addresses is indistinguishable from a deletion unless you say otherwise.

> A storage bucket is being handed to another department. It must keep existing with all its contents, Terraform must stop managing it, and the change must be reviewable in a pull request.

What should be written?

A. Delete the resource block and apply

B. Run terraform state rm on the bucket

C. A removed block with lifecycle destroy set to false

D. A moved block pointing at the other department's configuration

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should be written?

A Delete and apply

Terraform destroys what is no longer declared. The bucket and its contents would go.

B state rm

Achieves the outcome, immediately, from a terminal, with no plan and no review. The last requirement rules it out.

C removed, destroy = false

Correct. It forgets the binding, leaves the object alone, and appears in a plan so it goes through review.

D A moved block

moved renames within one configuration's state. It cannot move an object into a configuration owned by somebody else.

ON THE EXAM

Domain 6. Two options achieve the outcome. The requirement about review is what decides between them.

> One configuration is deployed to three environments, each with its own state bucket. An engineer adds a variable for the bucket name and references it from the backend block. terraform init fails.

What is the correct approach?

A Define the bucket name as a local value instead

B Move the backend block to its own file so it parses later

C Maintain three copies of the configuration

D Use partial configuration, supplying values at init time

◆ PAUSE AND DECIDE

What is the correct approach?

A Use a local

Locals are named values too. The restriction covers input variables, locals and data source attributes alike.

B A separate file

Filenames carry no meaning. Terraform reads all .tf files as one configuration, so ordering is unaffected.

C Three copies

Solves it by duplicating everything, reintroducing exactly the divergence problem modules exist to prevent.

D Partial configuration

Correct. A minimal backend block, with a per-environment settings file supplied via -backend-config at init.

ON THE EXAM

Domain 6. The restriction is about evaluation order, not syntax. The backend is resolved before variables, locals or data sources exist.

> An engineer needs to adopt one manually created resource quickly, as a one-off repair. A colleague says the terraform import command is deprecated and must not be used.

Is the colleague correct?

A No. The command is not deprecated and suits a one-off adoption

B Yes. It was deprecated when import blocks arrived in 1.5

C Yes. It was deprecated alongside taint and the refresh command

D No, but it only works for resources that support config generation

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Is the colleague correct?

A Not deprecated

Correct. import blocks are preferable for anything reviewed or repeated, and the command remains valid for a one-off.

B Deprecated in 1.5

1.5 added import blocks as a better option. Adding an alternative is not the same as deprecating the original.

C With taint and refresh

Those two genuinely are deprecated, which is what makes this attractive. The import command was not.

D Config generation only

Invents a restriction. Config generation is a separate feature of the block-based workflow.

ON THE EXAM

Domain 7. Paper one question 12 rewarded spotting an out-of-date pattern. This one punishes assuming that older always means wrong.

> A workspace must hold its own cloud credentials, its own list of who may apply, and a record of every run for audit.

Which kind of workspace provides this?

A A CLI workspace, created with terraform workspace new

B An HCP Terraform workspace

C Either. Both isolate credentials and permissions

D Neither. Credentials are always organisation-wide

◆ PAUSE AND DECIDE

Which kind of workspace provides this?

A. A CLI workspace

Isolates state and nothing else. Configuration, backend, credentials and permissions are all shared.

B. An HCP workspace

Correct. Its own variables, credentials, permissions, run history and state versions is the definition of one.

C. Either

The assumption the ambiguity exists to catch. HashiCorp calls the two distinctly different.

D. Neither

Overcorrects. HCP Terraform's permission model is built on exactly this separation.

ON THE EXAM

Domain 8. Decide which kind of workspace a question means before reading the options. The correct answers have nothing in common.

> A regulated team wants their Git repository to be the single source of truth. Every change must be reviewed as a pull request with a plan attached, and nobody should be able to start an apply from a workstation. They currently use the CLI-driven workflow with remote execution.

What should change?

A. Nothing. Remote execution already prevents applying from a workstation

B. Switch execution mode to Agent

C. Move to the VCS-driven workflow

D. Move to the API-driven workflow

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should change?

A Nothing needed

Remote execution moves where a run happens, not who can start one. An engineer can still type terraform apply.

B Agent mode

Execution mode again, answering a question about triggering. An agent changes where a run executes, not what starts it.

C VCS-driven workflow

Correct. Runs trigger from the repository, pull requests receive speculative plans, and the branch becomes the source of truth.

D API-driven

Requires building tooling for no benefit here, and approvals are available across workflows anyway.

ON THE EXAM

Domain 8. Workflow is what starts a run. Execution mode is where it runs. Two of these options answer the wrong question.

Scoring paper 2

- **The answer key, in order:** B D C A D B A C D B C A B D A C D A B C.
- **Three questions inverted paper one:** 6 against 20, 10 against 7, and 18 against 12.
- **Missing an inverted pair is the most useful miss on the paper.** It means you learned a mechanism as a rule.
- **Compare your domain misses with paper one.** A domain missed on both is a real gap.
- Paper three covers the remaining topics and closes by reading the trend across all three.

> ON THE EXAM
>
> A mechanism is never simply right or wrong. It fits a requirement or it does not, which is what the inverted pairs are for.